

Project #2

Class: CEN 4360

Semester: Fall 2025

Due Date: 2025-11-02

Design and Implementation of an Intermediate Android Application

With this assignment, students are tasked with designing an application and subsequently implementing it. The app functional requirements are provided, but the design / user interface can take any shape that supports the requirements. This app is aimed at helping students understand design practices, including rapid prototyping. In addition, it is aimed at reinforcing intermediate Android concepts, such as Fragments, Dialogues, and programmatically adding Views to Layouts. The learning outcomes for this assignment are as follows:

- Students will design an application with simple rapid prototyping.
- Students will implement an application according to their prototyping.
- Students will author specific business logic according to functional requirements.
- Students will author an application using the Model-View-Controller paradigm.

Students are expected to work alone on the assignment. You may discuss the assignment with others, but please try to complete the assignment by yourself before you seek help. This assignment is not meant to trick you in any way. If something is not clear, remember that you have resources available to you including my office hours and the TA's contact information. We want you to learn and succeed, so please reach out for any assistance!

A Classic Game on Android: Minesweeper

For this project, students will create an app that lets the user play Minesweeper. If you have not played Minesweeper, the game is available on Google (a simple search will suffice). The rules are simple:

- The game board is generated based on user preferences. It has X rows and Y columns.
- All cells start “uncovered”, meaning the player does not know where the mines exist.
- Some percentage of the cells are “mines”.
- When the player taps a cell that does not have a “Flag” (see below), they reveal its contents.
 - If it is a mine, the player loses.
 - If it is not a mine, reveal a number that tells how many mines immediately surround it.
 - If the number is 0, then immediately reveal all surrounding cells.
- When the player long-taps a cell, they suspect it as a mine. Put a “Flag” onto this cell, but keep it covered. Cells with a Flag cannot be revealed. To reveal it, the user must remove the flag by performing another long-tap on it.
- If the player reveals all non-mine cells, they win the game.
- If the player reveals a mine, they lose the game; show the player all other mine locations.

Note: this game has been around for many years. The purpose of this project is not to reinvent Minesweeper, but to instead implement it. I am certain you can find sample code online to help you. If you use an external source – which is permitted – please provide a link in your Java code comment to it.

App Requirements

In addition to implementing Minesweeper, the app will need to satisfy the following requirements.

The app must be separated into three main screens:

1. **Title / Instructions Screen** – this simple screen displays the title of the app, an eye-catching logo, and simple instructions for the game.
2. **Settings Screen** – this screen will allow the user to make choices for the game:
 - a. **Number of Rows** – options should include 5, 6, 7, 8, 9, and 10
 - b. **Number of Columns** – options should include 5, 6, 7, 8, 9, and 10
 - c. **% Mines** – options should include 10%, 15%, and 20%
 - d. **Cell Colors** – for each below, you can provide 5 different color selections for the user
 - i. **Covered Cell** (hasn’t been revealed yet)
 - ii. **Uncovered Cell** (has been revealed, and its not a mine)
 - iii. **Suspected Color** (hasn’t been revealed yet, but the player thinks it is a mine)
 - iv. **Mine Color** (the player revealed a mine, or all mines have been revealed)
3. **Game Screen** – when the player arrives at this screen, a gameboard using all of the user settings described above will be generated.

Facilitation of the multiple screens can be accomplished using Activities or Fragments.

Assume the phone will always be in Portrait mode; no need to handle phone rotations.

When the player wins or loses the game, display a Toast on the screen that indicates this.

Design Document Requirements

Along with the app, you must submit design documents for the app; specifically, you must create a rapid prototype that illustrates the different screens of the app. This should be created before you begin programming your app. A rapid prototype can take many forms, but for this project, please use a digital mockup. The mockup should illustrate how you envision the app to look on each screen.

Note: It is recommended to use a tool like Draw.io to complete your prototype; it has Android visuals to help you mockup an app. This app only has a few screens, so the design will be rather simple. This requirement is a “warmup” and the final project will require a more detailed design.

Submitting Your Assignment

1. Zip your entire project folder, which should include all XML, Gradle build scripts, Java, and image resources, into a single archive called "<your_last_name>_Project2.zip". In addition, place your Design documents inside the .zip.
 - a. For instance, mine would be called **Pfeil_Project2.zip**
2. In WebCourses, navigate to the assignment and use File Upload to submit your .zip.
3. If you do not submit your file on time, **you will receive a 0 for the assignment.**
4. If you submit the wrong file, **you will receive a 0 for the assignment.**
5. Your project must use Java (not Kotlin).
6. Late submissions will receive a penalty (see the Syllabus)!

Grading Your Assignment

Your assignment will be scored based on multiple factors, described below. Your grade will be posted to WebCourses. If you have any questions about your grade, please contact the TA; if you have further questions, please contact me. Any concerns about grades must be resolved within 1 week of the grade being posted.

Grading Criteria:

- **(30%) Prototype Design** – the project will be scored, in part, based on the quality and completeness of the rapid prototype provided. These should be completed in Draw.io and submitted as image files.
- **(60%) Functionality** – the project will be scored, in part, based on the completeness of the expected functionality. The user should be able to navigate through the three main screens of the app and play a complete game of Minesweeper, and the game board should be automatically generated based on the user's Settings inputs. A Toast should be used to show indicate win/lose.
- **(10%) Organization and Styling** – the project will be scored, in part, based on the solution's organization and styling. The code for this app should be organized using the MVC paradigm. The .java code should follow common styling guidelines, such as proper variable and method naming with camelCase, and provide adequate white space.